Introduction à Git et GitHub

Contrôle de version : Git, c'est le « Ctrl+Z sur les stéroïdes », avec la collaboration en plus grâce aux branches.

Soutien aux équipes distribuées : Git et GitHub permettent de contribuer depuis divers lieux sans conflits majeurs.

Favorise l'expérimentation : les branches permettent de développer des fonctionnalités sans perturber le code en production.

Pourquoi Git?



Architecture basée sur les snapshots : chaque commit capture l'état complet du projet.

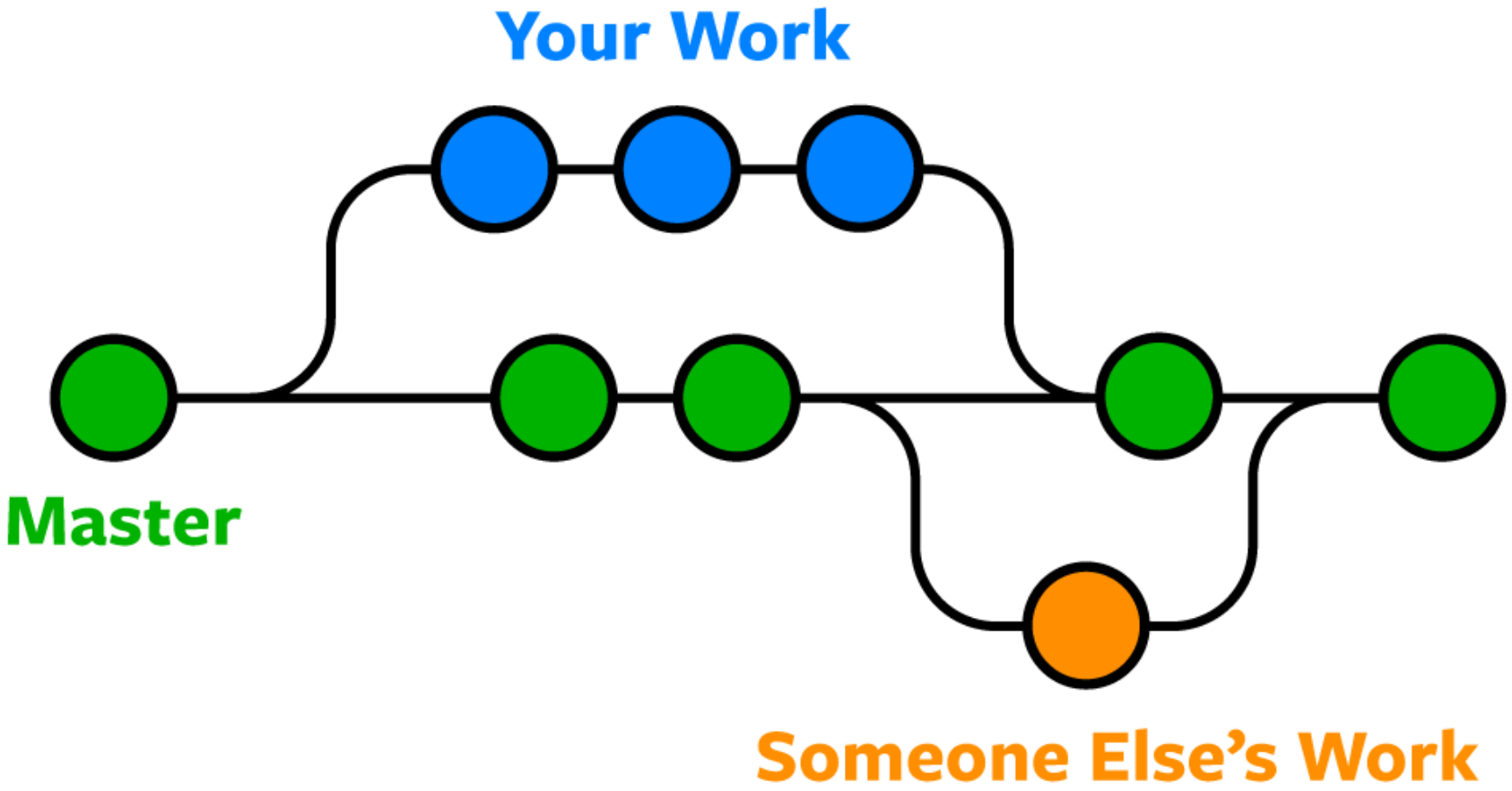


Efficacité des branches : légères et rapides à créer, idéales pour le développement en parallèle.



Adoption généralisée : plus de 87 % du marché mondial utilise Git.

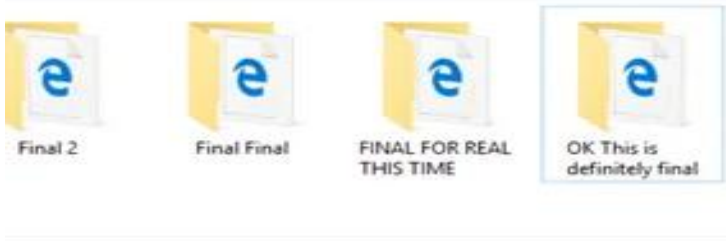
Comment fonctionne le système de branches



Git en bref


git


new.psd
newfinal.psd
newfinalfinal.psd
newfinalestfinal.psd
newfinalestfinal forsure.psd
newfinalestf kthis s itfinal.psd

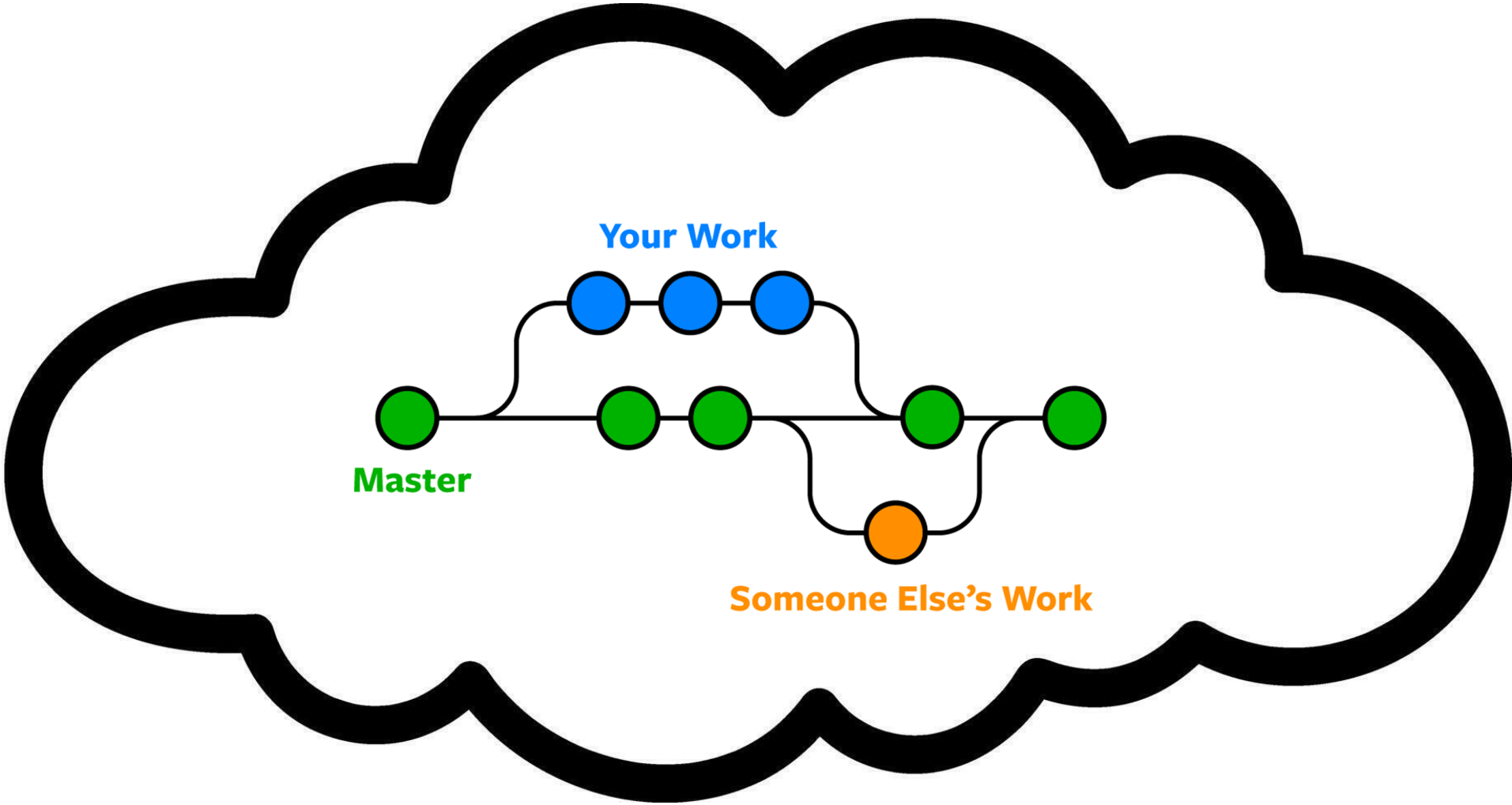

Final 2
Final Final
FINAL FOR REAL THIS TIME
OK This is definitely final


I prefer the real version control


I said the *real* version control


Perfection

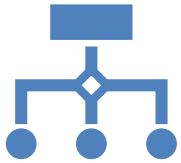
GitHub en bref



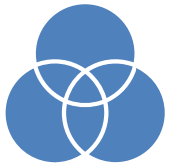
Bonnes pratiques Git & GitHub



Écrire un message clair à chaque commit pour s’y retrouver dans son historique.



Utiliser des branches : permet un développement modulaire.



Ne jamais pousser directement dans main : travailler dans sa propre branche et utiliser les pull requests.

Conclusion & Questions



Git : contrôle de version basé sur des snapshots



GitHub : héberge tes dépôts Git dans le cloud



Bonne pratique : JAMAIS de push direct dans main



Code along

Introduction aux dotfiles

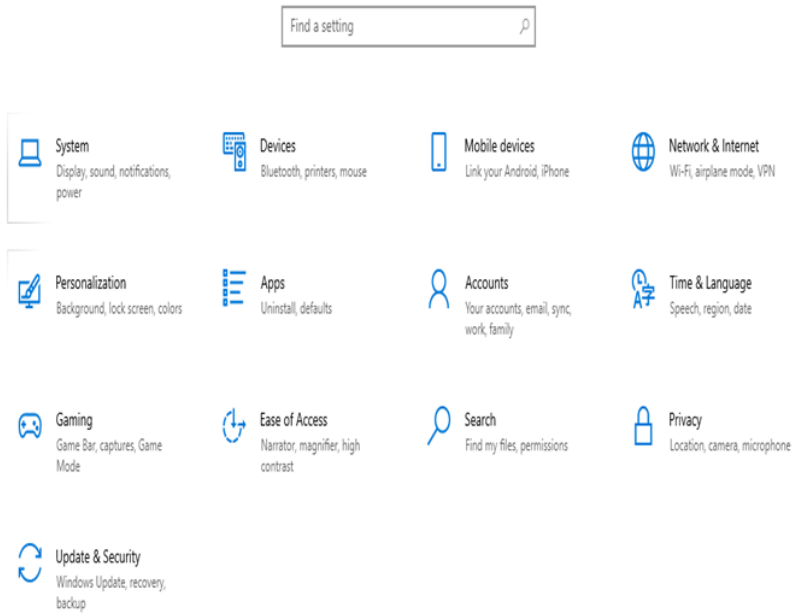


Avec Linux, tout est fichier

- Les réglages sont souvent des fichiers texte placés à des endroits précis
- Tu peux tout configurer, pas besoin d’interface graphique
- Cela inclut le shell, les éditeurs de texte, les outils système…








```yaml
#This file shows how to configure the Kube Agent Management Plugin
kube:
podTemplatesConfiguration:
  templates:
    - containers:
        - args: "cat"
          command: "/bin/sh -c"
          image: "nginx"
          livenessProbe:
            failureThreshold: 0
            initialDelaySeconds: 0
            periodSeconds: 0
            successThreshold: 0
            timeoutSeconds: 0
          name: "container0"
          ttyEnabled: true
          workingDir: "/home/jenkins/agent"
      hostNetwork: false
      name: "template0"
      yamlMergeStrategy: "override"
    - hostNetwork: false
      name: "template1"
      yamlMergeStrategy: "override"
```

.zshrc

- Définit :
  - les alias
  - le PATH vers tes scripts (bin)
  - les plugins (oh-my-zsh)
- Se trouve dans ton dossier personnel : ~/.zshrc

EN BREF

1) Configurer une seule fois dans le bon fichier (ex. : .zshrc)

2) Mettre ces configurations en contrôle de version avec Git

Avantages :

- Réutilisation sur d’autres systèmes
- Principe « only do it once »


Code along